

C++

程式語言（二）實習課

Introduction to Programming (II)

Practice Lessons

Templates

Joseph Chuang-Chieh Lin

Dept. CSE, NTOU

Platform/IDE

- Dev-C++



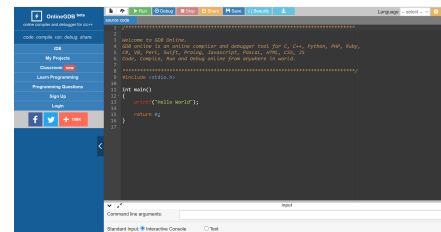
<https://www.pngegg.com/en/search?q=Dev-C>

- Codeblocks

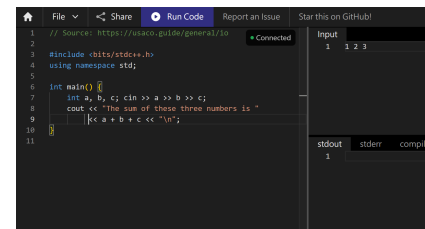


<https://icons8.com/icons/set/code-blocks>

- OnlineGDB (<https://www.onlinegdb.com/>)



- Real-Time Collaborative Online IDE (<https://ide.usaco.guide/>)



Useful Resources

- Tutorialspoint
 - <https://www.tutorialspoint.com/cplusplus/index.htm>
 - Online C++ Compiler
- Programiz
 - <https://www.programiz.com/cpp-programming>
- LEARN C++
 - <https://www.learncpp.com/>
- MIT OpenCourseWare - Introduction to C++
 - <https://ocw.mit.edu/courses/6-096-introduction-to-c-january-iap-2011/pages/lecture-notes/>
- Learning C++ Programming
 - <https://www.programiz.com/cpp-programming>
- GeeksforGeeks
 - <https://www.geeksforgeeks.org/c-plus-plus/>

Course TAs

- 林琮祐
 - Email address: 11357051@o365.ntou.edu.tw
- 張意苹
 - Email address: 11457070@o365.ntou.edu.tw
- 楊昀潼
 - Email address: 11457074@o365.ntou.edu.tw

Problem 1: Horner's Rule

Given the polynomial

$$p(x) = \sum_{i=0}^n a_i x^i = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots + a_n x^n,$$

where $a_0, a_1, \dots, a_n$ are constant coefficients,

the problem is to evaluate the polynomial at a specific value x_0 of x

Polynomial Evaluation

Given the polynomial

$$p(x) = \sum_{i=0}^n a_i x^i = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots + a_n x^n,$$

$n + (n - 1) + (n - 2) + \dots + 1 = (1 + n)n/2$ multiplications
and $n - 1$ additions



where $a_0, a_1, \dots, a_n$ are constant coefficients,

the problem is to evaluate the polynomial at a specific value x_0 of x

Reformulate the same polynomial:

Horner's Rule

$$\begin{aligned} p(x) &= \sum_{i=0}^n a_i x^i = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots + a_n x^n \\ &= a_0 + x (a_1 + x (a_2 + x (a_3 + \dots + x (a_{n-1} + x a_n) \dots))) \end{aligned}$$

n multiplications and *n* additions

For example,

$$\begin{aligned} p(x) &= 3 + 2x - x^2 + 6x^3 \\ &= 3 + x \cdot (2 + x \cdot ((-1) + x \cdot 6)). \end{aligned}$$

Reformulate the same polynomial:

Horner's Rule

$$\begin{aligned} p(x) &= \sum_{i=0}^n a_i x^i = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots + a_n x^n \\ &= a_0 + x (a_1 + x (a_2 + x (a_3 + \dots + x (a_{n-1} + x a_n) \dots))) \end{aligned}$$

n multiplications and *n* additions

```
int HornerRule(int coef[], int start, int degree, int x);

int HornersRule(int c[], int i, int d, int x) {
    if (i == d) return c[d];
    else
        return c[i] + x*HornerRule(c, i+1, d, x);
}
```

Horner's Rule

Consider the following program that evaluates a polynomial using Horner's Rule. You have a function `double horner(int poly[], int n, double x);` modify it so that `horner` becomes a function template and the program compiles, runs correctly.

```
double horner(int poly[], int n, double x) {  
    /* make it a template */  
}  
  
int main() {  
    int i, size;  
    double x;  
    cin >> size >> x;  
    double* poly = new double[size];  
    for (i=0; i<size; i++) {    cin >> poly[i];    }  
    cout << horner<double>(poly, size, x);  
    delete poly;  
    return 0;  
}
```

`poly[]`: $a_0 a_1 a_2 \dots a_{n-1}$

Sample Input	Sample output
4 3 2 -6 2 -1	-25
5 1 1 2 3 4 5	15
10 -1 1 2 3 4 5 6 7 8 9 10	-5

Problem 2: Sorting Students

Problem Statement

- In this exercise, you will implement both a class template and a function template.
 - First, define a class template `Student<ScoreType>`. The class stores a student's name and score. The score type should be generic, so we can create students whose scores are `int`, `double`, or other numerical types.
 - Then, define a function template `selection_sort<T>` that can sort an array of any type `T`, as long as a comparison function is provided.
 - Finally, use your sorting function to sort an array of `Student<int>` objects and an array of `Student<double>` objects by score in descending order.

Required Class Template

- Implement the following class template:

```
template <class ScoreType>
class Student {
private:
    string name;
    ScoreType score;

public:
    Student();
    Student(string n, ScoreType s);

    string getName() const;
    ScoreType getScore() const;

    void print() const;
};
```

- The function `print()` should print the student's name **and** score.

Required Function Template

- Implement the following function template:

```
template <class T>  
void selection_sort(T arr[], int n, bool (*before)(const T&, const T&));
```

- The function sorts the array according to the comparison function before.

`before(a, b)`

-> returns true, then a should appear before b.

Required Comparison Function Template

- Implement the following comparison function template:

```
template <class ScoreType>
bool higher_score(const Student<ScoreType>& a, const Student<ScoreType>& b);
```

- This function should return `true` if student `a` has a higher score than student `b`.

Main function (should NOT modify)

```
#include <iostream>
#include <string>
using namespace std;

// Implement Student<ScoreType> here.

// Implement selection_sort<T> here.

// Implement higher_score<ScoreType> here.
```

Sample output

```
Class A:
Bob 95
Amy 80
Cindy 70
Class B:
Eva 92.3
David 88.5
Frank 79.8
```

```
int main() {
    Student<int> classA[] = {
        Student<int>("Amy", 80),
        Student<int>("Bob", 95),
        Student<int>("Cindy", 70)
    };
    selection_sort<Student<int>>(classA, 3, higher_score<int>);
    cout << "Class A:" << endl;
    for (int i = 0; i < 3; i++) {
        classA[i].print();
    }
    Student<double> classB[] = {
        Student<double>("David", 88.5),
        Student<double>("Eva", 92.3),
        Student<double>("Frank", 79.8)
    };
    selection_sort<Student<double>>(classB, 3, higher_score<double>);
    cout << "Class B:" << endl;
    for (int i = 0; i < 3; i++) {
        classB[i].print();
    }
    return 0;
}
```